Computational Thinking

Introduction

Computational thinking is an approach to problem solving. Computational thinking is about taking a complex problem and being able to use several 'computation thinking techniques' (abstraction, decomposition, pattern recognition and algorithmic thinking) to:

- Fully understand the problem
- Create an effective solution.

This problem solving approach is not only important for programmers when devising software solutions, it is also an approach used in all areas of science to effectively enable complex processes to be understood.

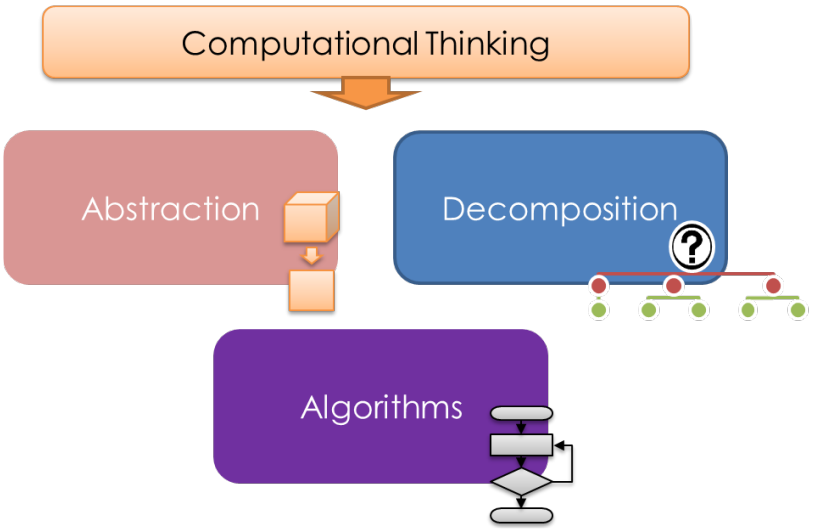
Can all problems be solved by a computer?

If a problem requires a software solution it must first be decided if it is solvable computationally. In other words, can the problem be solved by means of an algorithm? If so, then computational thinking techniques can be used. It must be noted that not all problems can be solved computationally.

Computational Thinking Techniques

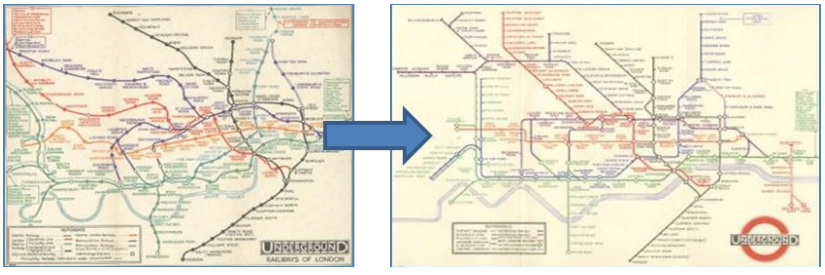
There are 3 main techniques that underpin computational thinking. They are:

- Abstraction
- Decomposition
- Algorithms



What is Abstraction?

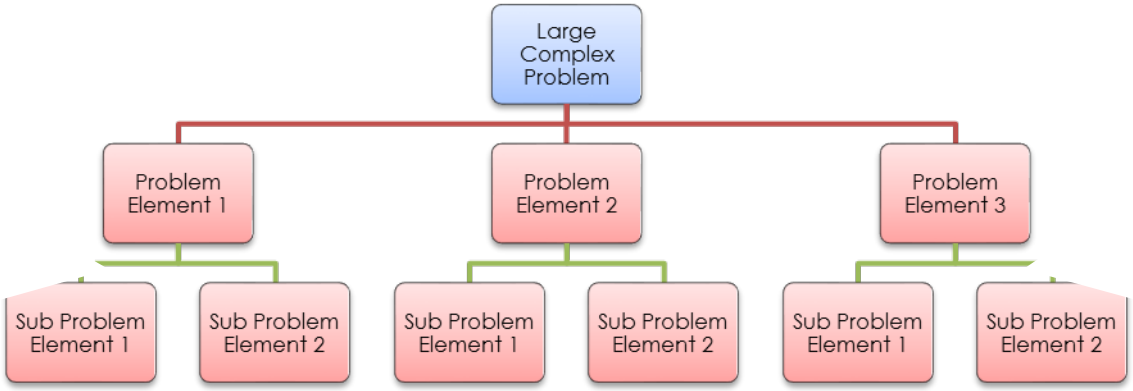
Abstraction is about moving from the real to virtual. It is concerned with simplifying the complex by only dealing with the important and neglecting the unnecessary. The London Underground map is a great example of Abstraction.



What is Decomposition?

Computer Science UK Membership Site Licence: Do not share outside your centre.

Decomposition is concerned with breaking down large complex problems into smaller problems that are far more easily solved. As you can see in the 'Top-Down' diagram below, a problem can be split up into its different elements. These elements can then be further split into sub problems which are more easily to solve.



What are Algorithms? What is Algorithmic Thinking?

Algorithms are simply recipes / procedures used to get the intended result. Algorithms state the steps required to get to the desired result. As we learnt earlier, for a problem to be computable, an algorithm must be produced. Creating algorithms is all about thinking logically and so to think computationally is to think in a structured, organised and logical manner. Algorithms are vital to programming. Without one, coding a solution is almost impossible...but with one, coding becomes an easy job (if you know your programming syntax that is!)

Questions *(The question zone you choose must either match your target grade or be higher!)*

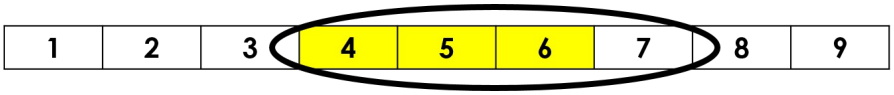
1	2	3	4	5	6	7	8	9

Computer Science UK Membership Site Licence: Do not share outside your centre.

Question Zone 1-3

1. State the names of 3 major computational thinking methods. [3]
2. What is the reason for decomposing a problem? [3]
3. What is 'Abstraction'? Use an example to back up your answer. [2]

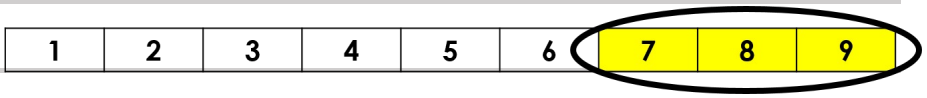
Question Zone 4-6



1. What is 'Abstraction'? Use an example to back up your answer. [2]
2. What is meant by the term 'Algorithmic Thinking'? [2]
3. "Mr W is in need of a program to help him with a problem he faces in his everyday life. Mr W likes running but always gets lost! He needs an app on his phone which will enable him to find his way home, no matter where he ends up, so that he can avoid the discomfort of sleeping outdoors. He hates sleeping outdoors! If he does he always wishes that he had a brought a sleeping bag. But carrying a sleeping bag when running is hard work! He wants his app to somehow point towards his home at all times during his run so he can find his way back!"

   Using abstraction, rewrite this problem by only stating what is important and neglect the irrelevant information. [6]

Questions Zone 7-9



1. "Mr W is in need of a program to help him with a problem he faces in his everyday life. Mr W likes running but always gets lost! He needs an app on his phone which will enable him to find his way home, no matter where he ends up, so that he can avoid the discomfort of sleeping outdoors. He hates sleeping outdoors! If he does he always wishes that he had a brought a sleeping bag. But carrying a sleeping bag when running is hard work! He wants his app to somehow point towards his home at all times during his run so he can find his way back!"

   Using abstraction, rewrite this problem by only stating what is important and neglect the irrelevant information. [6]
2. Using decomposition, create a top-down design tree diagram to help break this problem up into its component parts. [5]
3. Using 'algorithmic thinking', write an algorithm for one of the component parts (e.g. arrow to point home if he is not at home). [4]

Computer Science UK Membership Site Licence: Do not share outside your centre.

Checklist:

- Date and title, clearly presented
- Spelling & grammar checked
- Question numbers in the margin
- Handwriting neat & legible
- Punctuation / Capital letters

Keywords / Key Terms:

Abstraction: *The removal of irrelevant detail in order to focus on the problem at hand.*

Decomposition: *Breaking down problems into smaller sub-problems so that they are easier to solve.*

Algorithmic Thinking*: Thinking about the logical steps required to solve a problem*

Computer Science UK Membership Site Licence: Do not share outside your centre.

Stick answer sheet here

State/Identify/Give/Name: Simply label a diagram, fill in gaps, tick a box or write a few words
Describe: Describing is 'saying what you see' (E.G.: A computer will have a CPU, Primary and Secondary storage etc)
Explain: Explaining is 'saying WHY/HOW something is like that'. (E.G.: A computer will have a CPU so that it can process all of the data the computer needs to perform a range of tasks. Primary and Secondary storage is needed because...)
Discuss: Discussing is 'looking at two sides of an issue, weighing up the two views and giving a conclusion'. Often these require a mini essay answer. (E.G.: New technology could be seen as being bad for the environment because..., but on the other hand, new technology has led to... In conclusion I believe that...)
Describe/Explain/Discuss **using examples**: Finally, if you are asked to give examples in any of these types of questions – YOU MUST **GIVE EXAMPLES**!

Computer Science UK Membership Site Licence: Do not share outside your centre.

Reflections **Score:** / **Percentage:** % **Grade:** **Progress** On / Above / Below

What Went

- ☐ I demonstrated a good level of understanding.
- ☐ I responded to the command words effectively.
- ☐ My answers were detailed / were written in depth.
- ☐ My work was well presented / legible.
- ☐ My answers effectively incorporated technical terminology.
- ☐ My responses were well structured / organised.
- ☐ My revision strategy was effective as I showed depth of understanding in my answers.
- ☐ My answers contained enough points / examples / explanations to achieve the marks available.

Even Better

- ☐ My answers need to be more accurate.
- ☐ I must respond correctly to the command words.
- ☐ My answers need more detail / greater depth.
- ☐ I must take greater care over my work / write neatly.
- ☐ I must incorporate key terminology into my answers.
- ☐ I must better organise my answers to improve its clarity.
- ☐ I need to improve my revision strategy, as I did not demonstrate a depth of understanding in my answers.
- ☐ My answers didn't contain enough points / examples / explanations to achieve the marks available.

Further thoughts:

_____

_____

_____

_____

Computer Science UK Membership Site Licence: Do not share outside your centre.